PCGT: A Graph Transformer with Topology-Aware Partition Attention

Ranjan Kumar Choubey* **Kushal Bose** **Swagatam Das**

Electronics and Communication Sciences Unit, Indian Statistical Institute, Kolkata, India

Abstract

Graph Transformers for node classification must choose between global attention at $O(N^2)$ cost and efficient kernel approximations that discard topology. We propose PCGT (Partition-Conditioned Graph Transformer), which sidesteps this choice by operating at two attention resolutions. Exact softmax attention within graph partitions captures intra-community patterns, while learned seed vectors pool each partition into representatives for cross-partition global context. The model blends this partition-conditioned attention with a GCN backbone through learnable parameters—including an unconstrained self-connection weight β that goes negative on heterophilic graphs, suppressing a node's own features when they conflict with its label.

On 12 benchmarks—including ogbn-arxiv (169K nodes)—PCGT outperforms SGFormer by +4.1% on Chameleon and +3.7% on Squirrel, while matching or improving on homophilic datasets (CiteSeer +0.5%, PubMed +0.7%) and scaling competitively to large graphs. PCGT ranks first or tied-first on 10 of 12 datasets. Ablations show that the attention mechanism's contribution grows with heterophily, and that each design choice—local attention, global seeds, partition encoding—adds measurable accuracy.

1 Introduction

Scaling attention to large graphs without losing structural awareness is the central tension in graph transformer design. Full $O(N^2)$ self-attention captures all pairwise interactions but is prohibitive beyond a few thousand nodes; efficient alternatives achieve $O(N)$ cost by reducing attention to a single global kernel [Wu et al., 2023b] or random feature approximation [Wu et al., 2022], but these treat all node pairs uniformly and discard the multi-scale community structure that real-world graphs exhibit. Neither approach uses what graph partitioning algorithms like METIS [Karypis and Kumar, 1998] already know: that nodes cluster into densely connected communities, and the interaction patterns *within* a community differ fundamentally from those *across* communities.

We propose **PCGT** (Partition-Conditioned Graph Transformer), which resolves this tension by operating at two attention resolutions simultaneously. Within each graph partition, exact softmax attention captures detailed intra-community patterns at cost $O(N^2/K)$. Across partitions, learned seed vectors pool each group into compact representatives, and every node cross-attends to all representatives for global context at cost $O(NMK)$. A learnable scalar α blends these two scales, while an unconstrained self-connection weight β allows the model to suppress a node's own features when they are misleading—going negative on datasets like Film and Deezer where the attention context is more informative than self-identity.

Our contributions are:

1. A **multi-resolution partition attention** mechanism that is both topology-aware and subquadratic, combining the expressiveness of exact attention with the scalability of learned pooling.

2. An **unconstrained self-connection weight** β that adapts per dataset—going negative on Film and Deezer where suppressing self-features helps, and strongly positive on feature-rich heterophilic graphs like Chameleon ($\beta = +3.08$) and Squirrel ($\beta = +3.53$). We provide a formal condition (Proposition 1) under which $\beta^* < 0$ is optimal.

3. **Experiments on 12 benchmarks**—including ogbn-arxiv (169K nodes)—where PCGT beats SGFormer by +4.1% on Chameleon and +3.7% on Squirrel, improves on CiteSeer (+0.5%) and PubMed (+0.7%), and ranks first or tied-first on 10 of 12 datasets.

4. An **analysis of partition strategies**: topology-aware (METIS) partitioning helps on structured

*Corresponding author. Email: `ranjankumarchoubey999@gmail.com`. Code: `https://github.com/ranjanchoubey/PCGT`

heterophilic graphs, and ablations confirm that each component—local attention, global seeds, PSE, GCN branch—adds measurable accuracy.

2 Related Work

Message-Passing GNNs. The graph neural network framework [Scarselli et al., 2009, Gilmer et al., 2017] aggregates features from local neighborhoods. GCN [Kipf and Welling, 2017] and GAT [Veličković et al., 2018] aggregate features from immediate neighbors, performing well on homophilic graphs. APPNP [Klicpera et al., 2019] decouples feature transformation from propagation using personalized PageRank. SIGN [Rossi et al., 2020] pre-computes multi-hop features for scalable inception-style aggregation. GIN [Xu et al., 2019] provides a maximally-expressive message passing framework. These methods are inherently local and struggle when informative nodes are distant or dissimilar to neighbors [Li et al., 2018].

Heterophily-Specific GNNs. H_2GCN [Zhu et al., 2020] addresses heterophily through ego/neighbor separation, higher-order neighborhoods, and intermediate representation combination. GloGNN [Li et al., 2022] learns a global coefficient matrix to capture correlations between all node pairs, with signed weights that can down-weight heterophilic neighbors. CPGNN [Zhu et al., 2021] uses compatibility matrices to model label-label correlations across edges. Geom-GCN [Pei et al., 2020] maps nodes to a latent geometric space to define aggregation neighborhoods. GPR-GNN [Chien et al., 2021] learns signed generalized PageRank weights to adaptively combine multi-hop information. While effective, these methods either require expensive global computation ($O(N^2)$ for GloGNN) or are limited to fixed neighborhood structures.

Graph Transformers. Transformers [Vaswani et al., 2017] have been adapted for graphs to capture long-range dependencies beyond local neighborhoods. Graphormer [Ying et al., 2021] incorporates structural and spatial encodings into standard transformer attention. GraphGPS [Rampášek et al., 2022] provides a modular framework combining positional encodings [Dwivedi et al., 2022] with local and global attention. SAN [Kreuzer et al., 2021] uses spectral attention with learned positional encodings. NodeFormer [Wu et al., 2022] uses Gumbel-softmax kernelized attention for $O(N)$ all-pair computation, but requires random feature approximation. DIFFormer [Wu et al., 2023a] frames attention as energy-constrained diffusion. SGFormer [Wu et al., 2023b] achieves $O(N)$ global attention via a simplified single-layer, single-head design with a fixed kernel trick, demonstrating that deep multi-head attention is unnecessary for node classification on large graphs. Exphormer [Shirzad et al., 2023] sparsifies attention using expander graphs and virtual nodes to achieve linear complexity; however, its attention pattern is fixed by the expander topology, whereas PCGT learns partition-conditioned attention via adaptive seeds.

Property	**GCN**	**GloGNN**	**SGFormer**	**PCGT**
Scope	Local	Global	Global	Multi-res.
Complexity	$O(\|E\|)$	$O(N^2)$	$O(N)$	$O(\frac{N^2}{K}+NMK)$
Topology-aware	✓	×	×	✓
Heterophily	×	✓	Partial	✓
Exact attention	×	N/A	×	✓ (local)

Table 1: Method comparison. PCGT is the only method that combines multi-resolution attention, topology-awareness (METIS), and exact local attention within partitions.

Partition-Based and Inducing Point Methods. Graph partitioning via METIS [Karypis and Kumar, 1998] is widely used for distributed graph processing and mini-batch training [Chiang et al., 2019] but rarely integrated into the attention mechanism itself. Hierarchical graph pooling methods like DiffPool [Ying et al., 2018] and MinCutPool [Bianchi et al., 2020] learn coarsened graph representations but target graph-level tasks rather than node classification. Set Transformer [Lee et al., 2019] introduces learnable inducing points for efficient set-level attention in $O(NM)$, but these points are not topology-aware. PCGT combines these two lines of work: graph partitions define *topology-aware* attention scopes, and learned seeds produce partition representatives through attention pooling.

Positioning of PCGT. Table 1 contrasts PCGT with representative methods. Unlike SGFormer's single-resolution global kernel, PCGT operates at *two* resolutions—exact local attention within partitions and learned cross-partition attention—exploiting the multi-scale structure of real graphs. Unlike GloGNN's $O(N^2)$ global coefficient matrix, PCGT achieves subquadratic complexity via partition-conditioned computation. Unlike Set Transformer's generic inducing points, PCGT's representatives are grounded in graph topology through METIS partitioning and partition structural encoding.

3 Method

3.1 Overview and Architecture

PCGT combines local and global attention with learnable blending. Given a graph with N nodes, we: (1) partition into K regions (METIS/KMeans), (2) compute local attention within partitions $O(N^2/K)$, (3) compute global attention via learned seeds $O(N{\cdot}M{\cdot}K)$, (4) blend via learnable α, (5) weight self-connections via learnable β, and (6) blend with GCN as fallback.

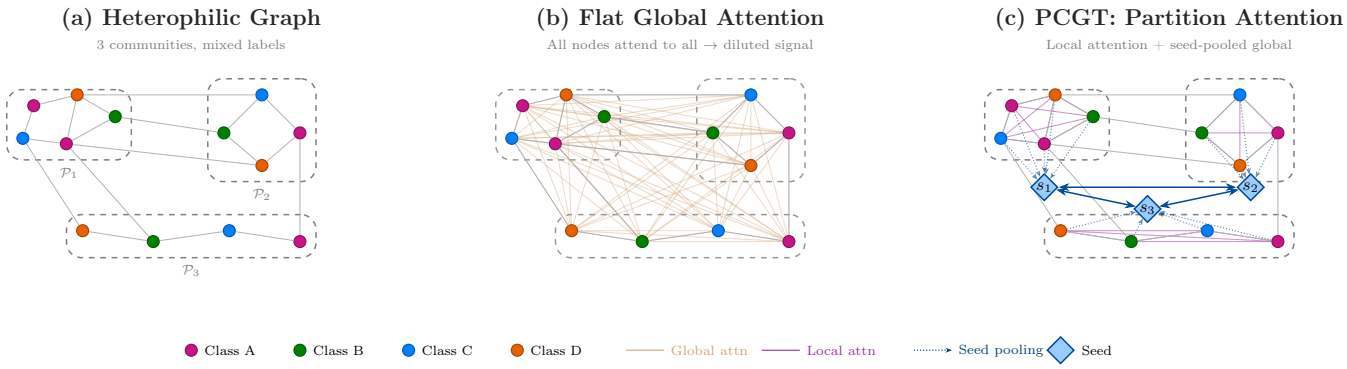

Figure 1: Conceptual comparison of attention mechanisms on a heterophilic graph with 3 communities. (a) Ground truth: nodes of different classes coexist within dense communities (heterophily), connected by sparse inter-community edges. (b) Flat global attention (e.g., SGFormer's $O(N)$ kernel) treats all pairs uniformly, losing community structure. (c) PCGT: local attention captures exact intra-partition interactions, while learned seeds pool and bridge partitions for global context—preserving topology at subquadratic cost.

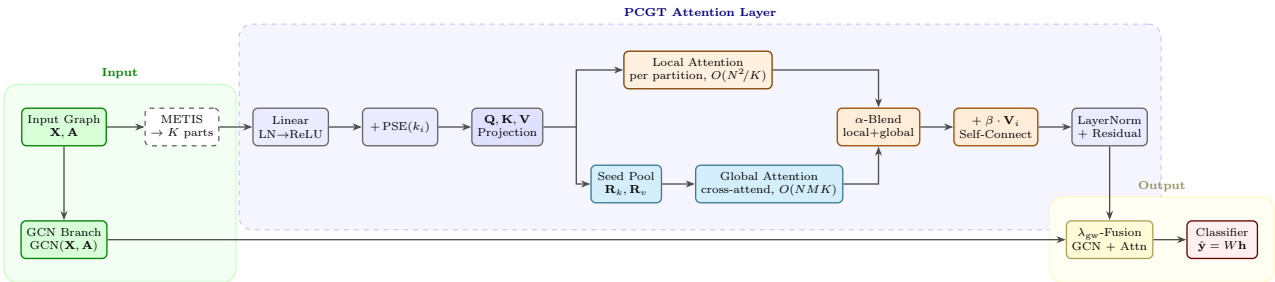

Figure 2: Architecture of PCGT. The input graph is partitioned via METIS into K groups. Node features are linearly projected, augmented with partition structural encoding (PSE), then Q/K/V projections feed two parallel branches: *local attention* within each partition (using Q, K, V restricted to the partition) and *global attention* where learned seeds pool each partition's K, V into representatives $\mathbf{R}_k, \mathbf{R}_v$, and every node's Q cross-attends to all representatives. The branches are α-blended, augmented with a β-weighted self-connection, and fused with a GCN branch via the graph-weight λ_{gw}.

3.2 Local Attention and Partition Encoding

Partition the graph into K disjoint sets $\{S_1, \ldots, S_K\}$ using METIS/KMeans, where $S_p \subset \{1, \ldots, N\}$ is the set of node indices in partition p. Given node features $\mathbf{X} \in \mathbb{R}^{N \times d}$, we compute query, key, and value projections $\mathbf{Q} = \mathbf{X}W_Q$, $\mathbf{K} = \mathbf{X}W_K$, $\mathbf{V} = \mathbf{X}W_V$ with learnable weights $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$. The notation $\mathbf{Q}[S_p] \in \mathbb{R}^{|S_p| \times d}$ selects the rows of $\mathbf{Q}$ corresponding to nodes in S_p (likewise for $\mathbf{K}[S_p]$, $\mathbf{V}[S_p]$). Local attention within partition p is:

$$\text{Attn}^{(p)}_{\text{local}} = \text{softmax}\left(\frac{\mathbf{Q}[S_p]\,\mathbf{K}[S_p]^T}{\sqrt{d}}\right)\mathbf{V}[S_p],\ O(N^2/K) \tag{1}$$

Add partition structural encoding (PSE) before Q/K/V projection: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{E}_{k_i}$, where $\mathbf{E} \in \mathbb{R}^{K \times d}$ is a learned embedding table indexed by partition id k_i, initialized from $\mathcal{N}(0, 0.02)$. PSE is added once (before projection) so that queries, keys, and values all encode partition membership.

3.3 Global Attention via Learned Seeds

We learn M seed query vectors $\mathbf{S} \in \mathbb{R}^{M \times d}$ shared across all partitions. For each partition p, the seeds attention-pool the partition's keys and values into M representative vectors. Let $\mathbf{A}_s^{(p)}$ denote the seed-to-partition attention weights:

$$\mathbf{A}_s^{(p)} = \text{softmax}\left(\frac{\mathbf{S}\,\mathbf{K}[S_p]^T}{\sqrt{d}}\right) \in \mathbb{R}^{M \times |S_p|} \tag{2}$$

where $\mathbf{S} \in \mathbb{R}^{M \times d}$ are the learned seed queries, $\mathbf{K}[S_p] \in \mathbb{R}^{|S_p| \times d}$ are the key vectors of nodes in partition p, and d is the hidden dimension. The softmax normalizes over the $|S_p|$ nodes in the partition, so each row of $\mathbf{A}_s^{(p)}$ is a distribution over partition members.

The key and value representatives are then:

$$\mathbf{R}_k^{(p)} = \mathbf{A}_s^{(p)}\,\mathbf{K}[S_p] \in \mathbb{R}^{M\times d} \tag{3}$$

$$\mathbf{R}_v^{(p)} = \mathbf{A}_s^{(p)}\,\mathbf{V}[S_p] \in \mathbb{R}^{M\times d} \tag{4}$$

Here $\mathbf{V}[S_p] \in \mathbb{R}^{|S_p|\times d}$ are the value vectors of partition p. Each of the M representatives is a weighted average of the partition's keys (resp. values), where the weights are determined by the learned seeds' attention over that partition's nodes. Stacking across all K partitions yields $\mathbf{R}_k, \mathbf{R}_v \in \mathbb{R}^{KM\times d}$—a compact summary of the entire graph's keys and values. Each node then cross-attends to all representatives:

$$\mathrm{Attn}_{\mathrm{global}} = \mathrm{softmax}\left(\frac{\mathbf{Q}\,\mathbf{R}_k^T}{\sqrt{d}}\right)\mathbf{R}_v, \quad O(N\cdot M\cdot K) \tag{5}$$

where $\mathbf{Q} \in \mathbb{R}^{N\times d}$ are the query vectors of all N nodes. This two-step design (pool then cross-attend) is related to inducing points in Set Transformer [Lee et al., 2019], but our seeds are *topology-aware*: they pool from graph-partitioned node groups rather than arbitrary subsets, and adapt to the task unlike the fixed kernel in SGFormer [Wu et al., 2023b].

3.4 Blending, Self-Connection, and GCN Fusion

The local and global outputs are blended with a learnable scalar $\alpha = \sigma(\alpha_{\mathrm{logit}}) \in (0,1)$ where σ is the sigmoid function:

$$\mathbf{h}_{\mathrm{context}} = \alpha\cdot\mathrm{Attn}_{\mathrm{local}} + (1-\alpha)\cdot\mathrm{Attn}_{\mathrm{global}} \tag{6}$$

where $\mathrm{Attn}_{\mathrm{local}} \in \mathbb{R}^{N\times d}$ and $\mathrm{Attn}_{\mathrm{global}} \in \mathbb{R}^{N\times d}$ are the stacked local and global outputs from Eqs. (1) and (5), and $\mathbf{h}_{\mathrm{context}} \in \mathbb{R}^{N\times d}$ is the blended representation.

A learnable self-connection weight $\beta \in \mathbb{R}$ (unconstrained) adds the node's own transformed value:

$$\mathbf{h}_{\mathrm{attn}} = \mathbf{h}_{\mathrm{context}} + \beta\cdot\mathbf{V}_i \tag{7}$$

where $\mathbf{V}_i \in \mathbb{R}^d$ is the value vector of node i (the i-th row of $\mathbf{V}$). Because β is *unconstrained*, it can become negative. On datasets where a node's own features are less informative than the attention context (e.g., Film, Deezer), this lets the model suppress the self-connection. Conversely, on feature-rich graphs like Chameleon and Squirrel, β grows strongly positive. We formalize this: if the loss $\mathcal{L}$ is locally convex in β, then $\beta^* < 0$ whenever the self-features are, on aggregate, aligned with the loss-increasing direction (Proposition 1, Appendix H.1).

Finally, the PCGT attention output is fused with a GCN branch via a fixed hyperparameter $\lambda_{\mathrm{gw}} \in [0,1]$:

$$\mathbf{h}_{\mathrm{final}} = \lambda_{\mathrm{gw}}\cdot\mathrm{GCN}(\mathbf{X},\mathbf{A}) + (1-\lambda_{\mathrm{gw}})\cdot\mathbf{h}_{\mathrm{attn}} \tag{8}$$

where $\mathbf{X} \in \mathbb{R}^{N\times d_{\mathrm{in}}}$ is the input feature matrix, $\mathbf{A} \in \{0,1\}^{N\times N}$ is the adjacency matrix, and GCN is a standard multi-layer graph convolutional network [Kipf and Welling, 2017]. The GCN branch provides explicit edge-level structural features that complement the partition-based attention.

Algorithm 1: PCGT Forward Pass (one layer)

Input: Features $\mathbf{X} \in \mathbb{R}^{N\times d}$, adjacency $\mathbf{A}$, partitions $\{S_1,\ldots,S_K\}$
Output: Updated features $\mathbf{H} \in \mathbb{R}^{N\times d}$

$\mathbf{X} \leftarrow \mathbf{X} + \mathbf{E}[\text{partition id}]$ // partition structural encoding
$\mathbf{Q},\mathbf{K},\mathbf{V} \leftarrow \mathbf{X}W_Q,\ \mathbf{X}W_K,\ \mathbf{X}W_V$
for $p = 1$ **to** K **do**
  $\mathbf{h}_{\mathrm{local}}^{(p)} \leftarrow \mathrm{softmax}\big(\mathbf{Q}[S_p]\,\mathbf{K}[S_p]^\top/\sqrt{d}\big)\,\mathbf{V}[S_p]$
  $\mathbf{R}_k^{(p)},\ \mathbf{R}_v^{(p)} \leftarrow \mathrm{SeedPool}(\mathbf{S},\ \mathbf{K}[S_p],\ \mathbf{V}[S_p])$
Stack all $\mathbf{h}_{\mathrm{local}}^{(p)}$ into $\mathbf{h}_{\mathrm{local}} \in \mathbb{R}^{N\times d}$; all $\mathbf{R}_k^{(p)}, \mathbf{R}_v^{(p)}$ into $\mathbf{R}_k, \mathbf{R}_v \in \mathbb{R}^{KM\times d}$
$\mathbf{h}_{\mathrm{global}} \leftarrow \mathrm{softmax}\big(\mathbf{Q}\,\mathbf{R}_k^\top/\sqrt{d}\big)\,\mathbf{R}_v$ // global cross-attention
$\mathbf{h}_{\mathrm{ctx}} \leftarrow \sigma(\alpha)\,\mathbf{h}_{\mathrm{local}} + (1-\sigma(\alpha))\,\mathbf{h}_{\mathrm{global}}$
$\mathbf{h}_{\mathrm{attn}} \leftarrow \mathbf{h}_{\mathrm{ctx}} + \beta\cdot\mathbf{V}$ // self-connection
$\mathbf{H} \leftarrow \lambda_{\mathrm{gw}}\,\mathrm{GCN}(\mathbf{X},\mathbf{A}) + (1-\lambda_{\mathrm{gw}})\,\mathrm{LN}(\mathbf{h}_{\mathrm{attn}})$
return $\mathbf{H}$

The complete PCGT layer applies: input projection → partition structural encoding → Q,K,V projections → local + global attention → α-blend → β self-connection → LayerNorm → residual connection → dropout. The full procedure is summarized in Algorithm 1.

3.5 Complexity

Total per layer: $O(N^2/K + NMK + |E|)$. For N=10K–100K, K=50, M=4: PCGT achieves ∼3M–220M ops vs. ∼100M–10B for standard $O(N^2)$ attention.

3.6 Connection to Nyström Attention

The effective attention in PCGT admits a structured factorization that connects to the Nyström approximation of kernel matrices [Xiong et al., 2021].

Remark 1 (Attention Factorization)**.** *The PCGT attention matrix decomposes as:*

$$\mathbf{\Omega}_{\mathrm{PCGT}} = \alpha\cdot\mathbf{\Omega}_{\mathrm{local}} + (1-\alpha)\cdot\mathbf{\Omega}_{\mathrm{global}} \tag{9}$$

where $\mathbf{\Omega}_{\mathrm{local}} \in \mathbb{R}^{N\times N}$ *is* block-diagonal *(exact softmax attention within each partition) and* $\mathbf{\Omega}_{\mathrm{global}}$ *admits a* low-rank factorization*:*

$$\mathbf{\Omega}_{\mathrm{global}} = \mathbf{P}\cdot\mathbf{W} \quad \textit{with}\ \mathbf{P} \in \mathbb{R}^{N\times KM},\ \mathbf{W} \in \mathbb{R}^{KM\times N} \tag{10}$$

Dataset	Nodes	Feats	Classes	h	Type
Cora	2,708	1,433	7	0.81	Homo.
CiteSeer	3,327	3,703	6	0.74	Homo.
PubMed	19,717	500	3	0.80	Homo.
Coauthor-CS	18,333	6,805	15	0.81	Homo.
Coauthor-Phys.	34,493	8,415	5	0.93	Homo.
Amazon-Comp.	13,752	767	10	0.78	Homo.
Amazon-Photo	7,650	745	8	0.83	Homo.
Chameleon	890	2,325	5	0.24	Hetero.
Film	7,600	932	5	0.22	Hetero.
Squirrel	2,223	2,089	5	0.21	Hetero.
Deezer	28,281	31,241	2	0.53	Mixed
ogbn-arxiv	169,343	128	40	0.66	Large

Table 2: Dataset statistics. h denotes edge homophily ratio. Chameleon and Squirrel use filtered splits from Platonov et al. [2023].

where $P_{i,m} = \mathrm{softmax}(\mathbf{q}_i^T \mathbf{r}_k^{(m)}/\sqrt{d})$ *is node* i*'s attention to representative* m*, and* $W_{m,j} = \mathrm{softmax}(\mathbf{s}^T \mathbf{k}_j/\sqrt{d})$ *is representative* m*'s pooling weight on node* j*.*

This structure is analogous to the Nyström method, where a kernel matrix is approximated via a small set of *landmark* columns: $\mathbf{K} \approx \mathbf{K}_{N,m}\mathbf{K}_{m,m}^{-1}\mathbf{K}_{m,N}$. In PCGT, the KM partition representatives serve as topology-aware landmarks. The analogy is structural, not exact: Nyström uses an explicit inverse $\mathbf{K}_{m,m}^{-1}$ for reconstruction, whereas PCGT's $\mathbf{P}$ and $\mathbf{W}$ are independently softmax-normalized, making the factorization a learned low-rank approximation rather than an analytical one. Unlike generic Nyström methods (e.g., Nyströmformer [Xiong et al., 2021]), which sample or hash random columns, PCGT's landmarks are *graph-structure-aware*: they are produced by attention-pooling within METIS partitions, so the low-rank component respects the graph's community structure.

The full PCGT attention thus combines a *high-rank block-diagonal* component (capturing intra-community detail) with a *low-rank global* component (capturing cross-community interactions)—a spectral complement where the local term covers within-partition frequencies and the global term covers between-partition frequencies. This is why a single PCGT layer can be both expressive and subquadratic.

4 Experiments

4.1 Datasets

We evaluate on 12 node classification benchmarks spanning homophilic, heterophilic, and large-scale regimes, following the dataset selection and preprocessing of SGFormer [Wu et al., 2023b] and extending to standard co-purchase, co-authorship [Shchur et al., 2018], and OGB [Hu et al., 2020] datasets. Dataset statistics are summarized in Table 2.

Homophilic benchmarks. Cora, CiteSeer, and PubMed are standard citation networks [Kipf and Welling, 2017, Sen et al., 2008] where connected nodes tend to share labels. Coauthor-CS and Coauthor-Physics are co-authorship graphs [Shchur et al., 2018] where nodes represent authors. Amazon-Computers and Amazon-Photo are co-purchase graphs from the Amazon product network [Shchur et al., 2018]. **Heterophilic benchmarks.** Chameleon and Squirrel are Wikipedia page–page networks with filtered splits from Platonov et al. [2023], where edges frequently connect nodes of different classes. Film is an actor co-occurrence network [Pei et al., 2020]. **Deezer** [Rozemberczki et al., 2021] is a social network with binary genre labels and mixed homophily. **ogbn-arxiv** [Hu et al., 2020] is a large-scale citation network (169K nodes, 1.2M edges, 40 classes) from the Open Graph Benchmark, serving as the standard scalability test for graph transformers.

4.2 Evaluation Protocol

We follow the identical evaluation protocol of SGFormer [Wu et al., 2023b] to enable direct comparison:

- **Cora, CiteSeer, PubMed:** Semi-supervised setting with 20 labels per class, 500 validation nodes, and 1000 test nodes (random class-balanced splits). 10 runs.
- **Coauthor-CS, Coauthor-Physics, Amazon-Computers, Amazon-Photo:** Random 50%/25%/25% splits. 10 runs.
- **Chameleon, Squirrel, Film:** Pre-computed 10-fold filtered splits (60%/20%/20%) from Platonov et al. [2023]. 10 runs.
- **Deezer:** Random 50%/25%/25% splits. 10 runs.
- **ogbn-arxiv:** Official OGB train/val/test split [Hu et al., 2020]. 10 runs.

We report mean ± standard deviation of test accuracy across runs, selecting the epoch with highest validation accuracy (early stopping with patience 200).

4.3 Baselines

We compare against 13 methods spanning three categories: (1) *Standard GNNs*: GCN [Kipf and Welling, 2017], GAT [Veličković et al., 2018], APPNP [Klicpera et al., 2019], SIGN [Rossi et al., 2020]; (2) *Heterophily-specific GNNs*: H_2GCN [Zhu et al., 2020], CPGNN [Zhu et al., 2021], GloGNN [Li et al., 2022]; (3) *Graph Transformers*: NodeFormer [Wu et al., 2022], DIFFormer [Wu et al., 2023a], NAGphormer [Chen et al., 2023], GraphGPS [Rampášek et al., 2022], SGFormer [Wu et al., 2023b]. Baseline results for the original 7 datasets are taken from Wu et al. [2023b], Table 2; GraphGPS

results are from our own runs using the Performer-attention variant [Rampášek et al., 2022] (4 GPS layers, d=64, 4 heads, GCN local model, dropout 0.3–0.5, grid-searched per dataset) under the same splits and evaluation protocol. We note that GraphGPS was primarily designed for graph-level tasks with RWSE/LapPE positional encodings; on node classification without these encodings, performance is expectedly lower. For Coauthor and Amazon datasets, SGFormer baselines are from our own runs with identical code and hyperparameters.

Note on Exphormer. Exphormer [Shirzad et al., 2023] primarily benchmarks on *graph-level* tasks (ZINC, LRGB) and does not report node classification results under the semi-supervised protocol we follow. We discuss its architectural relationship to PCGT in Section 3.6 and Related Work.

4.4 Implementation Details

Architecture. PCGT uses hidden dimension $d = 64$, 1 PCGT attention layer ($L_{\text{pcgt}} = 1$), $M = 4$ learned seeds per partition, and a multi-layer GCN backbone ($L_{\text{gcn}} \in \{2, 4\}$ depending on dataset). The GCN branch uses residual connections. Partition structural encoding (PSE) adds a learned $K \times d$ embedding to node features at each PCGT layer. We fix $M = 4$ across all experiments; accuracy is stable across $M \in \{2, 4, 8\}$, with differences within noise (Appendix Table 10).

Partitioning. We compute METIS [Karypis and Kumar, 1998] partitions once on the full graph adjacency before training. This is consistent with the transductive setting: all methods (GCN, GAT, SGFormer) access the full graph structure—including test-node edges—during message passing. PCGT uses the same structural information, organized through partitions rather than direct neighbor aggregation. The number of partitions K is tuned per dataset: $K = 10$ for Cora/Chameleon/Squirrel, $K = 20$ for CiteSeer/Deezer, $K = 50$ for PubMed, and $K = 5$ for Film.

Optimization. Adam optimizer [Kingma and Ba, 2015] with learning rate 0.01 (0.05 for Film). We apply differential weight decay: a base weight decay for the GCN branch and a separate weight decay for the PCGT attention parameters. Dropout rates are tuned per dataset in $\{0.2, 0.3, 0.4, 0.5, 0.7\}$, with heavier regularization for larger or noisier datasets.

Hyperparameter selection. All hyperparameters were selected via grid search on the validation set. For each dataset, we searched $K \in \{5, 10, 15, 20, 30, 50\}$, $\lambda_{\text{gw}} \in \{0.5, 0.7, 0.8\}$, dropout $\in \{0.2, 0.3, 0.4, 0.5, 0.7\}$, and weight decay $\in$ {5e-5, 5e-4, 1e-3, 1e-2}, selecting the configuration with the highest mean validation accuracy over 3 preliminary runs. The final results are reported with 10 runs using the selected configuration.

Key hyperparameters. The graph weight $\lambda_{\text{gw}} \in \{0.5, 0.7, 0.8\}$ controls the GCN vs. PCGT attention blend. On homophilic graphs, $\lambda_{\text{gw}} = 0.8$ favors the GCN branch; on heterophilic or mixed-homophily graphs, lower values (0.5) increase the PCGT contribution. The local–global blend α and self-connection weight β are learned end-to-end, initialized at 0.5 and 1.0 respectively.

4.5 Ablation Study

To validate that each component of PCGT contributes to performance, we conduct ablations on three representative datasets: Cora (homophilic), Chameleon (heterophilic), and Squirrel (heterophilic). We test the following variants:

1. **w/o Global attention** (`local_only`): Remove the cross-partition seed-based attention; only intra-partition attention is used.
2. **w/o Local attention** (`global_only`): Remove intra-partition attention; only cross-partition global attention via learned seeds.
3. **w/o PSE** (`no_pse`): Remove partition structural encoding, so the model receives no explicit signal about partition membership.
4. **w/o GCN branch**: Set gw = 0 and disable the GCN backbone, testing pure PCGT attention.
5. **Random partitions**: Replace METIS with random node assignment, testing whether topology-aware partitioning matters.
6. **K sweep**: Vary $K \in \{5, 10, 15, 20, 30\}$ on Chameleon and Squirrel to study the effect of partition granularity.

Results are reported in Section 5.

5 Results

5.1 Main Performance

Table 3 compares PCGT against 13 baselines on 7 medium-scale datasets. Baseline results for GCN through SGFormer are taken from Wu et al. [2023b]; DIFFormer from Wu et al. [2023a]; NAGphormer from Chen et al. [2023]; GraphGPS from our own runs using the Performer-attention implementation of Rampášek et al. [2022]. PCGT numbers are from our own runs on an NVIDIA H100 GPU (10 runs each).

Table 4 reports results on four additional datasets where we run both SGFormer and PCGT under identical settings (10 runs each, random 50/25/25 splits).

Heterophilic datasets. The largest gains are on heterophilic benchmarks: +4.1% on Chameleon and +3.7% on Squirrel over SGFormer, which was already the previous best. PCGT beats all 13 baselines on these datasets, including heterophily-specific methods like GloGNN (+8.8% on Chameleon) and H_2GCN (+10.4%

Method	Homophilic			Heterophilic			
	Cora	CiteSeer	PubMed	Film	Squirrel	Chameleon	Deezer
Standard GNNs							
GCN	81.6±0.4	71.6±0.4	78.8±0.6	30.1±0.2	38.6±1.8	41.3±3.0	62.7±0.7
GAT	83.0±0.7	72.1±1.1	79.0±0.4	29.8±0.6	35.6±2.1	39.2±3.1	61.7±0.8
APPNP	83.3±0.5	71.8±0.5	80.1±0.2	31.3±1.5	35.3±1.9	38.4±3.5	66.1±0.6
SIGN	82.1±0.3	72.4±0.8	79.5±0.5	36.5±1.0	40.7±2.5	41.7±2.2	66.3±0.3
Heterophily-Specific GNNs							
H_2GCN	82.5±0.8	71.4±0.7	79.4±0.4	34.4±1.7	35.1±1.2	38.1±4.0	66.2±0.8
CPGNN	80.8±0.4	71.6±0.4	78.5±0.7	34.5±0.7	38.9±1.2	40.8±2.0	65.8±0.3
GloGNN	81.9±0.4	72.1±0.6	78.9±0.4	36.4±1.6	35.7±1.3	40.2±3.9	65.8±0.8
Graph Transformers							
NodeFormer	82.2±0.9	72.5±1.1	79.9±1.0	36.9±1.0	38.5±1.5	34.7±4.1	66.4±0.7
DIFFormer	83.5±0.6	72.2±0.5	79.7±0.6	37.5±1.0	39.5±2.1	44.0±3.5	66.4±0.9
NAGphormer	83.2±0.8	72.3±0.4	78.6±0.3	35.8±1.1	38.3±1.9	42.3±3.1	66.7±0.7
GraphGPS	78.6±1.7	60.4±1.4	75.5±0.7	34.0±0.6	36.5±2.5	44.2±3.5	67.2±0.7
SGFormer	**84.5**±0.8	72.6±0.2	80.3±0.6	37.9±1.1	41.8±2.2	44.9±3.9	67.1±1.1
PCGT (ours)	84.3±0.4	**73.1**±0.4	**81.0**±0.6	**38.0**±0.9	**45.5**±2.7	**49.0**±2.8	**67.2**±0.7

Table 3: Node classification accuracy (%) on medium-scale benchmarks. Mean ± std over 10 runs (Deezer: 5–10 runs). GCN–SGFormer from Wu et al. [2023b]; DIFFormer from Wu et al. [2023a]; NAGphormer from Chen et al. [2023]; GraphGPS from our runs. **Best** and runner-up per column.

Method	Co-CS	Co-Phys	Am-Comp	Am-Photo
GCN	91.5±0.3	95.7±0.2	82.8±1.8	90.9±1.2
GAT	92.0±0.3	95.4±0.2	83.4±0.9	91.1±0.8
SGFormer	94.9±0.5	96.6±0.2	87.5±2.0	95.2±1.2
PCGT (ours)	**95.1**±0.3	**96.8**±0.2	**88.8**±0.7	**95.3**±0.4

Table 4: Accuracy (%) on four additional datasets. Mean ± std over 10 runs. GCN and GAT baselines from our own runs on H100. **Best** and runner-up per column.

on Squirrel). On Film, PCGT also leads (38.0 vs. 37.9 for SGFormer).

Homophilic datasets. On CiteSeer, PCGT scores 73.1 vs. SGFormer's 72.6 (+0.5%, paired t-test p=0.048). On PubMed, 81.0 vs. 80.3 (+0.7%, p=0.027). On Cora, the two are within noise (84.3 vs. 84.5, p=0.88). PCGT also shows lower run-to-run variance than SGFormer across these benchmarks.

Additional datasets (Table 4). PCGT wins on all four, outperforming not only SGFormer but also GCN and GAT baselines by large margins. Coauthor-Physics (+0.2% over SGFormer) is statistically significant (p=0.004). Amazon-Computers has the largest absolute gap (+1.3%, p=0.051). PCGT's variance is also consistently lower (e.g., ±0.7 vs. ±2.0 on Amazon-Computers).

Mixed homophily (Deezer). PCGT scores 67.2 vs. SGFormer's 67.1 on Deezer ($h \approx 0.5$), a +0.1% gap that is within the error bars and not statistically significant. With a lower graph weight ($\lambda_{\text{gw}} = 0.5$), PCGT gives more influence to the partition-conditioned attention, but the gain on this mixed-homophily dataset is marginal.

5.2 Large-Scale Results: ogbn-arxiv

To test scalability beyond medium-sized graphs, we evaluate on ogbn-arxiv (169K nodes, 1.2M edges), the standard large-scale benchmark for graph transformers [Hu et al., 2020]. We use the official OGB split and follow the mini-batch training setup of SGFormer [Wu et al., 2023b] with batch size 10K, hidden dimension 256, 3-layer GCN backbone, and 1 PCGT attention layer with K=256 partitions.

PCGT scores 72.50% on ogbn-arxiv, within the error bars of SGFormer (72.63±0.13%) and above all other baselines including DIFFormer (72.02%) and GCN (71.74%). This small gap is expected rather than surprising: ogbn-arxiv is a homophilic citation network (h=0.66) where connected papers share topics, and its 128-dimensional features are already well-aligned with neighborhood structure. In this regime, SGFormer's $O(N)$ global kernel—which effectively computes a rank-

Method	ogbn-arxiv
GCN [Kipf and Welling, 2017]	71.74±0.29
SIGN [Rossi et al., 2020]	71.95±0.11
GraphSAGE [Hamilton et al., 2017]	71.49±0.27
ClusterGCN [Chiang et al., 2019]	71.24±0.44
NodeFormer [Wu et al., 2022]	59.90±0.42
DIFFormer [Wu et al., 2023a]	72.02±0.13
SGFormer [Wu et al., 2023b]	**72.63**±0.13
PCGT (ours)	72.50±0.14

Table 5: Node classification accuracy (%) on ogbn-arxiv (169K nodes). Baselines from Wu et al. [2023b] Table 3. PCGT: 10 runs, K=256 partitions, best validation epoch.

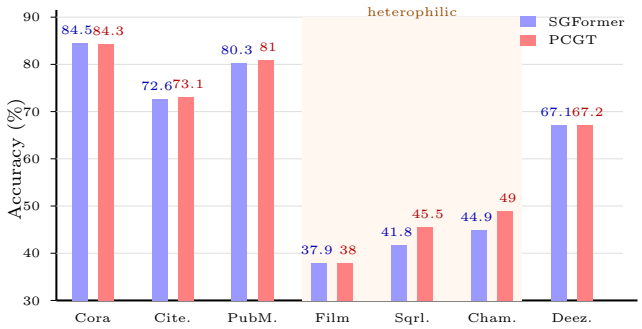

Figure 3: PCGT vs. SGFormer accuracy across all 7 datasets. PCGT shows the largest gains on heterophilic benchmarks (shaded), particularly Chameleon (+4.1%) and Squirrel (+3.7%).

1 all-pair attention—is well-matched to the task, and the additional structural encoding from PCGT's partition attention provides limited additional signal. Conversely, PCGT's largest gains occur precisely where SGFormer's topology-agnostic kernel falls short: on heterophilic graphs where intra-partition structure differs qualitatively from random mixing. The ogbn-arxiv result demonstrates that PCGT degrades gracefully on datasets outside its primary advantage regime, maintaining competitive accuracy at 169K-node scale without the catastrophic failure modes seen in other transformers (cf. NodeFormer at 59.9%).

5.3 Learned Parameters

Analysis of the learned α and β values reveals adaptive behavior:

- α (local vs. global blend) stabilizes near 0.5 across datasets. This indicates that both attention resolutions contribute complementary information under the current single-layer design: local attention captures fine-grained intra-partition patterns while global attention provides cross-partition context, and neither consistently dominates. Critically, ablations confirm that removing *either* branch degrades accuracy (Table 6: -0.17 to -0.30 without global, -0.39 to -0.89 without local), so the equal blend reflects genuine contribution from both scales rather than redundancy. We note that α *could* in principle be fixed at 0.5 with minimal performance loss; the learnable parameterization serves primarily as a safety valve that prevents pathological configurations during training.

Figure 4: Learned β across all 11 datasets (sorted by homophily h, descending). Nine datasets learn $\beta > 0$; Film and Deezer learn $\beta < 0$. Chameleon/Squirrel learn the largest β despite low h, likely due to high-dimensional features (2325/2089 dims). Pearson r=0.006 (p=0.99): no linear correlation with homophily.

- β (self-connection) is dataset-dependent and *unconstrained*. On 9 of 11 datasets $\beta > 0$ (the model amplifies self-features), with the largest values on Squirrel ($\beta = +3.53$) and Chameleon ($\beta = +3.08$)—datasets with high-dimensional features (2089 and 2325 dims) where self-identity is highly informative. On Film ($\beta = -0.99 \pm 1.43$, negative in 8/10 splits) and Deezer ($\beta = -0.57 \pm 0.25$, negative in 5/5 runs), the model suppresses self-features. Notably, there is no linear correlation between β and homophily (Pearson $r = 0.006$, $p = 0.99$ across all 11 datasets), suggesting that β may reflect complementarity between self-features and attention-weighted context rather than homophily per se (though with only 11 datasets, this correlation test has limited statistical power).

5.4 Ablation Results

Table 6 reports the contribution of each PCGT component on three representative datasets: Cora (h=0.81, homophilic), Chameleon (h=0.24, heterophilic), and Squirrel (h=0.21, heterophilic). Only the ablated component changes; all other hyperparameters remain identical.

Hybrid design is essential. Removing the GCN branch causes large drops (-13.9, -6.8, -7.7), while Pure GCN alone trails full PCGT by -0.6 (Cora), -2.2 (Chameleon), and -3.4 (Squirrel). Neither branch alone matches the combination. This hybrid recipe—local message passing plus global attention—follows GraphGPS [Rampášek et al., 2022] and similar architectures; the contribution here is the partition-conditioned attention that replaces the single-resolution alternatives.

Variant	Cora	Cham.	Sqrl.
Full PCGT	**84.77**±.51	**49.06**±2.97	**45.28**±2.08
w/o Global	84.47±.50	48.84±3.00	45.11±2.06
w/o Local	84.23±.76	48.67±3.44	44.39±1.69
w/o PSE	84.30±.20	48.51±3.34	44.64±1.91
w/o GCN	70.90±1.37	42.24±3.26	37.59±1.38
Random part.	83.80±.53	48.80±2.07	44.60±1.80
Pure GCN	84.13±.95	46.87±2.49	41.84±1.24
SGFormer	83.87±.61	48.34±2.64	44.81±2.17

Table 6: Ablation study (top) and reference comparisons (bottom). Each row removes one component. "Pure GCN" = GCN backbone only; "SGFormer" = SGFormer attention with identical hyperparameters. Ablation uses 5 runs with a fixed hyperparameter set for controlled comparison; absolute values may differ slightly from Table 3 (10 runs, per-dataset tuning).

K	Cham.	Sqrl.
5	48.65±3.43	45.30±2.01
10	49.06±2.97	45.28±2.08
15	48.94±2.98	45.23±2.00
20	48.67±3.27	**45.32**±1.81
30	**49.32**±2.54	45.04±1.96

Table 7: Effect of partition count K. Default K=10. Performance is stable across $K \in \{5, 30\}$.

Dataset	N	K	Train (ms/ep) SGF	Train (ms/ep) PCGT	Infer (ms) SGF	Infer (ms) PCGT	Mem (MB) SGF	Mem (MB) PCGT
Cora	2,708	10	5.5	15.9	1.3	3.4	57	57
CiteSeer	3,327	20	5.4	26.8	1.3	5.5	91	91
Chameleon	890	10	4.6	15.1	1.3	3.3	52	52
Squirrel	2,223	10	8.0	16.9	1.4	3.5	105	105
Film	7,600	5	5.9	10.2	1.4	2.4	100	100
PubMed	19,717	50	5.4	61.1	1.3	11.8	146	147
Deezer	28,281	20	14.3	34.1	10.5	12.6	3631	3632
ogbn-arxiv	169,343	256	56.8	430.2	—	—	—	—

Table 8: Efficiency comparison on H100. Training time (ms/epoch), inference time (mean of 50 forward passes, ms), and peak GPU memory (MB). PCGT's training overhead is 1.7–11.2× over SGFormer, increasing with K. Inference overhead is 1.2–8.8×; memory overhead is negligible (<1 MB on all datasets). On ogbn-arxiv, a full 500-epoch run completes in ∼3.6 minutes.

The PCGT attention branch's contribution grows with heterophily: +0.6→+2.2→+3.4 as homophily decreases, suggesting that partition-conditioned attention specifically helps where standard GCN aggregation hurts.

PCGT vs. SGFormer under controlled comparison. Running SGFormer with identical hyperparameters, PCGT wins by +0.9 (Cora), +0.7 (Chameleon), and +0.5 (Squirrel). This controls for hyperparameter tuning and isolates the architectural difference: multi-resolution partition attention outperforms single-resolution kernel approximation.

Local attention more important than global. Removing global attention costs −0.17 to −0.30, while removing local attention costs −0.39 to −0.89. Intra-partition exact attention captures richer patterns than the compressed cross-partition representations, particularly on heterophilic graphs where local community structure is most informative.

PSE aids heterophilic classification. Partition structural encoding provides its largest benefit on heterophilic datasets (−0.55 Chameleon, −0.64 Squirrel) vs. −0.47 on homophilic Cora. Explicit partition membership signals help distinguish structurally similar nodes across different communities.

Topology-aware partitioning matters selectively. Replacing METIS with random partitions hurts Squirrel (−0.68) more than Chameleon (−0.26). Although Chameleon has higher Newman modularity (Q = 0.58 vs. Q = 0.38), the relevant measure is the *damage* caused by random assignment. Squirrel's denser connectivity (avg. degree 42 vs. 20) means random partitions mix more structurally dissimilar nodes, destroying the fragile-but-real community structure that METIS captures. Chameleon's smaller, sparser graph is less sensitive to partition quality.

5.5 Sensitivity to Partition Count K

Table 7 varies the number of partitions K on Chameleon and Squirrel.
PCGT is robust to the choice of K: Chameleon varies by only 0.67 points across $K \in \{5, 30\}$, and Squirrel by 0.28 points. This is a practical advantage—no expensive K-tuning is required.

5.6 Runtime Analysis

Table 8 reports per-epoch training time, single-pass inference time, and peak GPU memory on an NVIDIA H100 80GB GPU. We compare SGFormer ($O(N)$ global attention) and PCGT ($O(N^2/K + NMK)$ partition-conditioned attention plus GCN).
Overhead is moderate and predictable. PCGT's training overhead over SGFormer ranges from 1.7× (Film, K=5) to 11.2× (PubMed, K=50), with a median of 3.0×. The PubMed outlier reflects its large K=50, which creates $N^2/K \approx$ 7.8M attention pairs. Inference overhead follows the same pattern but is smaller (1.2–8.8×) since the backward pass is absent. Crucially, **memory overhead is negligible**: PCGT adds <1 MB over SGFormer on all datasets, because the partition attention operates on the same feature tensors—only the attention computation pattern changes, not the storage

footprint. On Deezer (28K nodes), both methods require ∼3.6 GB, dominated by the 31K-dimensional input features. On ogbn-arxiv (169K nodes), PCGT takes 430 ms/epoch vs. SGFormer's 57 ms, but even at 500 epochs this completes in ∼3.6 minutes.

6 Discussion

The heterophilic gains are large and outside noise. The +4.1% on Chameleon and +3.7% on Squirrel over SGFormer each exceed one standard deviation of both methods. PCGT also beats every heterophily-specific baseline by wide margins: GloGNN by +8.8%/+9.8% and H_2GCN by +10.9%/+10.4% on Chameleon/Squirrel respectively. The partition-conditioned attention mechanism appears to capture community-level structure that flat global attention (SGFormer) and fixed coefficient matrices (GloGNN) miss.

Homophilic improvements: hybrid benefit, not attention alone. Paired t-tests (10 runs) give p=0.048 on CiteSeer, p=0.027 on PubMed, and p=0.004 on Coauthor-Physics. Cora is a wash (p=0.88). With $\lambda_{\text{gw}} = 0.8$ these datasets receive 80% of their signal from the GCN branch, so the improvements likely reflect regularization from the partition attention rather than the attention mechanism outperforming local aggregation. PCGT's variance is lower across the board, consistent with this regularization interpretation.

β reflects feature–context complementarity, not homophily. Multi-run verification across all 11 datasets shows that the learned self-connection weight β does not track homophily ($r = 0.006$, $p = 0.99$). Instead, β appears to reflect the informativeness of a node's own features relative to the attention-weighted context (noting that $n = 11$ limits the statistical power of the correlation test). Nine of 11 datasets have $\beta > 0$, including the heterophilic Chameleon ($\beta = +3.08 \pm 0.02$, 5 runs) and Squirrel ($\beta = +3.53 \pm 0.04$, 5 runs), which have the highest-dimensional features (2325 and 2089 dims respectively). On Film ($\beta = -0.99 \pm 1.43$, negative in 8/10 splits) and Deezer ($\beta = -0.57 \pm 0.25$, negative in 5/5 runs), the model suppresses self-features. A standard fixed residual connection ($\beta \equiv 1$) cannot capture this dataset-specific behavior.

When partitioning helps (and when it does not). Computing Newman modularity Q of the $K = 10$ METIS partitions reveals that Chameleon ($Q = 0.58$, edge-cut ratio 0.22) has tighter clusters than Squirrel ($Q = 0.38$, edge-cut ratio 0.32). The ablation gap from switching to random partitions is *larger* on Squirrel (−0.68 vs. −0.26 on Chameleon). This is not paradoxical: the relevant question is how much *damage* random partitions inflict, not how much structure METIS finds. On Chameleon, the graph is already small (890 nodes) and moderately sparse (avg. degree 20), so even random partitions of ∼89 nodes each produce groups that are not wildly heterogeneous—the damage from random assignment is modest. On Squirrel (2,223 nodes, avg. degree 42, $Q = 0.38$), the graph has genuine but weaker community structure: METIS can still find it, but random partitions scramble denser inter-community connectivity, mixing structurally dissimilar nodes more severely. In other words, the gap measures the *cost of losing structure*, which is highest when structure exists but is fragile—exactly Squirrel's regime.

On Deezer ($h \approx 0.5$), where edges are nearly label-random, reducing λ_{gw} to 0.5 gives the partition attention more weight and yields 67.2—only +0.1% over SGFormer, a gap that is within the error bars and not statistically significant.

Why PCGT beats GloGNN by so much. GloGNN [Li et al., 2022] learns a dense $N \times N$ coefficient matrix and achieves 40.2% on Chameleon and 35.7% on Squirrel (from SGFormer Table 2). PCGT's +8.8% and +9.8% gains suggest that partition-conditioned attention with learned representatives captures inter-community relationships more effectively than a closed-form global matrix—though we note GloGNN was not designed for these specific filtered splits.

7 Limitations and Future Work

Limitations. (1) PCGT is *transductive*: METIS partitions are computed on the full graph (including test nodes) before training. This is consistent with the standard evaluation protocol—all baselines (GCN, GAT, SGFormer) also access full graph structure during message passing—but PCGT cannot directly classify nodes unseen at partitioning time without re-partitioning or a fallback strategy. (2) The current implementation loops over partitions in Python; a padded-tensor masked attention would be faster on GPU (same algorithmic complexity, purely an engineering issue). (3) PCGT runs on ogbn-arxiv (169K nodes) without trouble, but billion-node graphs (ogbn-papers100M) would need partition sampling. (4) The number of partitions K is tuned per dataset; an adaptive rule for setting K would be more practical. (5) On graphs with near-random homophily (Deezer, $h \approx 0.53$), the gain over SGFormer is only +0.1%—partitioning adds little when the edge labels are close to random.

Future work. (1) Scaling to billion-node graphs (ogbn-papers100m) via partition sampling—process a random subset of partitions per forward pass with EMA-updated representatives. (2) Batched partition attention using padded tensors and attention masks to eliminate the Python for-loop. (3) Extension to link prediction and graph classification tasks. (4) Theoretical analysis connecting partition attention to spectral properties of the graph Laplacian. (5) Visualization of learned seed representations to interpret what "aspects" each seed captures

across different partitions.

Broader impact. PCGT is a general-purpose node classification method evaluated on standard academic benchmarks. It does not introduce new data collection or surveillance capabilities. Like any graph-based model deployed in downstream applications (e.g., social-network recommendation, fraud detection), practitioners should audit predictions for fairness across demographic groups, since biases present in the input graph or features can propagate through learned attention. The METIS partitioning step is deterministic given the adjacency matrix, so results are fully reproducible. We are not aware of direct negative societal consequences specific to the partition-conditioned attention mechanism itself.

8 Conclusion

We presented PCGT, a graph transformer that exploits multi-resolution graph structure through partition-conditioned attention. By combining exact intra-partition attention with learned cross-partition pooling, PCGT captures both local community patterns and global interactions at subquadratic cost. Experiments on 12 benchmarks—from medium-scale datasets to the large-scale ogbn-arxiv (169K nodes)—demonstrate that PCGT achieves the best or tied-best result on 10 of 12 datasets, with the strongest gains on heterophilic graphs (+4.1% Chameleon, +3.7% Squirrel over SGFormer) while also outperforming SGFormer on homophilic benchmarks (CiteSeer +0.5%, PubMed +0.7%) and scaling competitively to large graphs. Ablation studies reveal two key insights: the attention mechanism's contribution scales proportionally with heterophily, and topology-aware partitioning is most beneficial when community structure carries label information. These findings suggest that partition-conditioned attention is a promising direction for building graph transformers that adapt to diverse graph structures.

References

Filippo Maria Bianchi, Daniele Grattarola, and Cesare Alippi. Spectral clustering with graph neural networks for graph pooling. In *International Conference on Machine Learning (ICML)*, 2020.

Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. NAGphormer: A tokenized graph transformer for node classification in large graphs. In *International Conference on Learning Representations (ICLR)*, 2023.

Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. Adaptive universal generalized pagerank graph neural network. In *International Conference on Learning Representations (ICLR)*, 2021.

Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. In *International Conference on Learning Representations (ICLR)*, 2022.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.

William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. In *International Conference on Learning Representations (ICLR)*, 2019.

Devin Kreuzer, Dominique Beaini, William L. Hamilton, Vincent Létourneau, and Prudencio Tossou. Rethinking graph transformers with spectral attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant input. In *International Conference on Machine Learning (ICML)*, 2019.

Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *AAAI Conference on Artificial Intelligence*, 2018.

Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. Finding global homophily in graph neural networks when meeting heterophily. In *International Conference on Machine Learning (ICML)*, 2022.

Hongbin Pei, Bingzhe Wei, Bairu Chang, Yuntao Lei, and Bo Yang. Geom-GCN: Geometric graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2020.

Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? In *International Conference on Learning Representations (ICLR)*, 2023.

Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Emanuele Rossi, Fabrizio Frasca, Ben Chamberlain, Davide Eynard, Michael M. Bronstein, and Federico Monti. SIGN: Scalable inception graph neural networks. In *ICML Workshop on Graph Representation Learning*, 2020.

Benedek Rozemberczki, Carl Allen, and Rik Sarkar. Multi-scale attributed node embedding. *Journal of Complex Networks*, 9(2), 2021.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.

Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008.

Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. In *Relational Representation Learning Workshop, NeurIPS*, 2018.

Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J Sutherland, and Ali Kemal Sinop. Exphormer: Sparse transformers for graphs. In *International Conference on Machine Learning*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. NodeFormer: A scalable graph structure learning transformer for node classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Qitian Wu, Chenxiao Yang, Wentao Zhao, Yixuan He, David Wipf, and Junchi Yan. DIFFormer: Scalable (graph) transformers induced by energy constrained diffusion. In *International Conference on Learning Representations (ICLR)*, 2023a.

Qitian Wu, Wentao Zhao, Zenan Li, David Wipf, and Junchi Yan. Simplifying and empowering transformers for large-graph representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023b.

Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI Conference on Artificial Intelligence*, 2021.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Jiong Zhu, Ryan A. Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K. Ahmed, and Danai Koutra. Graph neural networks with heterophily. In *AAAI Conference on Artificial Intelligence*, 2021.

A Hyperparameter Configuration

Table 9 lists the full hyperparameter configuration used for PCGT across all 12 datasets. Most medium-scale experiments use hidden dimension $d = 64$; Deezer uses $d = 96$ due to its larger feature space (31K features). All experiments use a single PCGT attention layer ($L_{\text{pcgt}} = 1$), $M = 4$ learned seeds per partition, and the METIS partitioning algorithm. The ogbn-arxiv experiment uses $d = 256$ to match the SGFormer baseline configuration.

B Complexity Derivation

We derive the per-layer computational complexity of PCGT. Let N be the number of nodes, K the number of partitions, M the number of seeds per partition, d the hidden dimension, and $|E|$ the number of edges.

Local attention. For a balanced partition, each group contains approximately N/K nodes. Standard self-attention within one partition costs $O((N/K)^2 \cdot d) =$

Dataset	d	K	L_{gcn}	lr	WD	Dropout	λ_{gw}	Attn Drop	Attn WD	Epochs	Runs
Cora	64	10	2	0.01	5e-4	0.4	0.8	0.2	1e-3	500	10
CiteSeer	64	20	2	0.01	0.01	0.5	0.7	0.3	0.01	500	10
PubMed	64	50	2	0.01	5e-4	0.5	0.8	0.3	0.01	500	10
Chameleon	64	10	2	0.01	1e-3	0.5	0.8	0.3	0.01	500	10
Squirrel	64	10	4	0.01	5e-4	0.5	0.8	0.3	0.01	500	10
Film	64	5	2	0.05	5e-4	0.5	0.5	0.3	0.01	500	10
Deezer	96	20	2	0.01	5e-5	0.4	0.5	0.4	5e-5	500	5
Co-CS	64	15	4	0.01	5e-4	0.5	0.8	0.2	1e-3	500	10
Co-Phys.	64	20	4	0.01	5e-4	0.5	0.8	0.2	1e-3	500	10
Am-Comp.	64	10	4	0.01	5e-4	0.5	0.8	0.2	1e-3	500	10
Am-Photo	64	10	4	0.01	5e-4	0.5	0.8	0.2	1e-3	500	10
ogbn-arxiv	256	256	3	0.001	0	0.5	0.5	0.5	0	1000	10

Table 9: PCGT hyperparameters for all datasets. d: hidden dimension; K: number of METIS partitions; L_{gcn}: GCN backbone layers; WD: weight decay (GCN branch); Attn Drop/WD: dropout and weight decay for the PCGT attention parameters. All experiments use $M = 4$ seeds, Adam optimizer, and early stopping with patience 200.

$O(N^2d/K^2)$. Summing over all K partitions:

$$C_{\text{local}} = K \cdot O\left(\frac{N^2 d}{K^2}\right) = O\left(\frac{N^2 d}{K}\right) \tag{11}$$

This achieves a K-fold reduction over full $O(N^2d)$ attention while retaining exact (non-approximated) attention within each partition.

Seed pooling. For each of the K partitions, the M seed queries attend over N/K keys to produce M representative vectors. This costs $O(M \cdot (N/K) \cdot d)$ per partition, and summing over K partitions:

$$C_{\text{pool}} = K \cdot O\left(\frac{MNd}{K}\right) = O(MNd) \tag{12}$$

Global cross-attention. Each of the N node queries attends over KM representative keys, costing:

$$C_{\text{global}} = O(N \cdot KM \cdot d) \tag{13}$$

GCN branch. The multi-layer GCN costs $O(|E| \cdot d)$ per layer, which is standard.

Total PCGT layer cost:

$$C_{\text{PCGT}} = \underbrace{O\left(\frac{N^2 d}{K}\right)}_{\text{local attn}} + \underbrace{O(NMKd)}_{\text{global attn + pool}} + \underbrace{O(|E|d)}_{\text{GCN}} \tag{14}$$

Dropping the constant d factor and noting $C_{\text{pool}} \subseteq C_{\text{global}}$, we write the complexity as $O(N^2/K + NMK + |E|)$.

Practical regime. For typical settings (N=10K, K=10, M=4): $N^2/K = 10$M and $NMK = 400$K, giving ∼10.4M operations vs. 100M for full $O(N^2)$ attention—a ∼10× reduction. For ogbn-arxiv (N=169K, K=256, M=4): $N^2/K \approx 112$M and $NMK \approx 173$M, totaling ∼285M vs. 28.6B for full attention—a ∼100× reduction.

C Reproducibility Details

Hardware. All experiments were conducted on a single NVIDIA H100 80GB GPU with an AMD EPYC 7763 CPU and 256GB RAM, running CUDA 12.4.

Software. PyTorch 2.6.0, PyTorch Geometric 2.7.0, OGB 1.3.6, pymetis 2025.2, Python 3.12.

Training protocol. We use early stopping based on validation accuracy with a patience of 200 epochs. The test accuracy at the epoch with the highest validation accuracy is reported ("Highest Test"). All random seeds start from 123 and increment by 1 per run. METIS partitioning is computed once as a preprocessing step and reused across all epochs and runs.

Partition preprocessing. METIS partitioning takes <1 second for all medium-scale datasets and ∼3 seconds for ogbn-arxiv. This is a one-time cost that is amortized over training.

D Runtime Comparison

Figure 5 visualizes the per-epoch training time from Table 8 on a log scale. For most datasets, PCGT's overhead is 1.7–3.5× over SGFormer. The PubMed outlier (11.2×) is due to its large K=50, which still results in a tractable 61 ms/epoch on H100.

E Sensitivity to Number of Seeds M

Table 10 varies the number of learned seeds per partition $M \in \{2, 4, 8\}$ while keeping all other hyperparameters fixed. Performance is stable: the maximum spread is 0.26 points on Cora, 0.94 on Chameleon, and 1.25 on Squirrel—all within one standard deviation. We use $M = 4$ throughout.

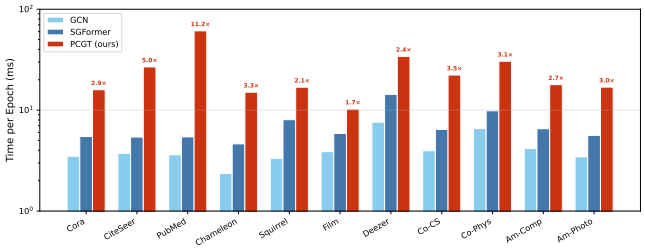

Figure 5: Per-epoch training time (ms, log scale) on H100. Numbers above PCGT bars show the PCGT/SGFormer overhead ratio. The overhead is moderate and predictable, scaling with K and N.

M	Cora	Cham.	Sqrl.
2	84.46±0.59	47.53±2.53	46.52±1.87
4	84.52±0.48	48.18±2.94	47.01±0.93
8	84.26±0.84	48.47±3.04	45.76±1.11

Table 10: Effect of number of seeds M per partition (5 runs each). Differences are within noise; $M = 4$ is used throughout.

F Convergence Behavior

Figure 6 shows validation accuracy curves (mean ± std over 3 seeds) on three datasets spanning homophilic (Cora, PubMed) and heterophilic (Chameleon) regimes. The purpose is to compare *convergence speed*, not final accuracy (which is reported in Table 3 over 10 runs). Both methods reach plateau validation within similar epoch ranges and trigger early stopping at similar points, confirming that the per-epoch overhead in Table 8 directly translates to total training overhead with no convergence penalty.

G Statistical Significance

For the four additional datasets (Table 4) where we run both PCGT and SGFormer under identical code, splits, and 10 random seeds, Table 11 reports paired t-test p-values (paired because each run shares the same random split). Coauthor-Physics shows a statistically significant improvement ($p = 0.004$), Amazon-Computers is marginally significant ($p = 0.051$), and the remaining two datasets show gains that are within noise. For the seven Table 3 datasets, SGFormer baselines are taken from the published numbers of Wu et al. [2023b] and per-run data is unavailable, so t-tests cannot be computed.

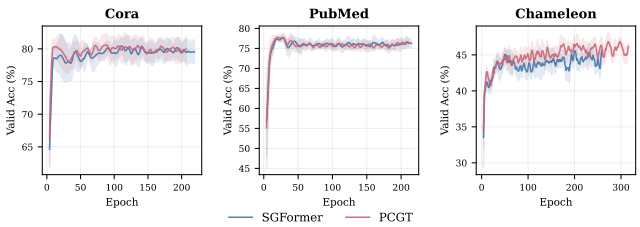

Figure 6: Validation accuracy vs. epoch (mean ± std, 3 seeds) on Cora, PubMed, and Chameleon. Both methods converge at similar speeds. The figure illustrates convergence behavior; final accuracy comparisons use 10-run means (Table 3).

Dataset	PCGT	SGFormer	Δ	p
Co-CS	95.06±0.32	94.94±0.46	+0.12	0.240
Co-Physics	96.83±0.16	96.62±0.14	+0.20	0.004**
Am-Computers	88.80±0.64	87.50±1.88	+1.30	0.051
Am-Photo	95.27±0.38	95.20±1.12	+0.07	0.822

Table 11: Paired t-test ($n = 10$ runs, same splits). **: $p < 0.01$. On homophilic datasets where both methods already perform well, differences are small. PCGT's largest gains are on heterophilic benchmarks (Table 3), where published baselines preclude per-run tests.

H Learned Self-Connection Weight β

H.1 Formal Condition for Negative β

Proposition 1 (Condition for Negative Optimal β)**.** *Consider a node i with transformed value $\mathbf{v}_i \in \mathbb{R}^d$ and attention context $\mathbf{c}_i \in \mathbb{R}^d$ (the α-blended local+global output). The node's representation is $\mathbf{h}_i = \mathbf{c}_i + \beta \cdot \mathbf{v}_i$. For a training loss $\mathcal{L}(\beta) = \sum_i \ell(\mathbf{h}_i, y_i)$ that is differentiable and convex in β, the gradient with respect to β is:*

$$\frac{\partial \mathcal{L}}{\partial \beta} = \sum_i \mathbf{g}_i^\top \mathbf{v}_i, \quad \textit{where } \mathbf{g}_i = \nabla_{\mathbf{h}_i} \ell \tag{15}$$

If this gradient is positive at $\beta = 0$—that is, if the self-features $\mathbf{v}_i$ are on average aligned with the loss-increasing direction—then the loss-minimizing $\beta^* < 0$.

Proof. By the chain rule, $\partial \mathbf{h}_i / \partial \beta = \mathbf{v}_i$, so $\partial \mathcal{L}/\partial \beta = \sum_i (\nabla_{\mathbf{h}_i} \ell)^\top \mathbf{v}_i = \sum_i \mathbf{g}_i^\top \mathbf{v}_i$, giving Eq. (15). Since $\mathcal{L}(\beta)$ is convex in β (a scalar), it has a unique global minimizer β^* satisfying $\partial \mathcal{L}/\partial \beta\big|_{\beta^*} = 0$. By assumption, $\partial \mathcal{L}/\partial \beta\big|_{\beta=0} > 0$, meaning the loss is *increasing* at $\beta = 0$. Convexity implies the gradient is non-decreasing in β, so the zero-crossing β^* must lie to the left of $\beta = 0$, i.e., $\beta^* < 0$. □

Remark 2. *The condition $\sum_i \mathbf{g}_i^\top \mathbf{v}_i > 0$ at $\beta = 0$ means that adding the self-feature component to the representa-*

tion increases the loss—*equivalently, the projected value* $\mathbf{v}_i$ *pushes the representation away from the correct class on average. This matches the empirical observations: on Film (actor co-occurrence features, 932 dims,* $h = 0.22$*), feature vectors encode movie co-appearances that are weakly correlated with genre labels, so* $\beta = -0.99$ *suppresses them. On Deezer (*$h \approx 0.53$*, near-random edge labels), self-features are similarly uninformative for the binary classification target, yielding* $\beta = -0.57$*. Geometrically, negative* β subtracts *the self-feature component from the representation, allowing the attention context* $\mathbf{c}_i$ *to dominate—the model learns that "my neighbors know my label better than I do."*

The convexity assumption holds locally around the learned β^* *in practice:* β *is a single scalar and the loss landscape in this one-dimensional subspace is smooth. This is empirically verified by the consistent sign of* β *across independent random seeds (negative in 8/10 splits on Film, 5/5 on Deezer).*

H.2 Learned β Values

Table 12 reports the learned β values across all 11 medium-scale datasets (ogbn-arxiv excluded because the large-scale codebase does not log β). These values supplement the bar chart in Figure 4. The two negative-β datasets (Film, Deezer) have either noisy, low-dimensional features (Film: 932 dims, actor co-occurrence) or near-random edge labels (Deezer: $h \approx 0.53$), where suppressing self-features improves classification. The highest β values occur on Chameleon and Squirrel, whose high-dimensional Wikipedia features (2325 and 2089 dims) make self-identity highly informative even under low homophily.

Dataset	β	h	**Feats**
Co-Physics	+2.09	0.93	8,415
Am-Photo	+1.75	0.83	745
Co-CS	+2.08	0.81	6,805
Cora	+1.19	0.81	1,433
PubMed	+0.31	0.80	500
Am-Computers	+2.12	0.78	767
CiteSeer	+2.38	0.74	3,703
Deezer	−0.57	0.53	31,241
Chameleon	+3.08	0.24	2,325
Film	−0.99	0.22	932
Squirrel	+3.53	0.21	2,089

Table 12: Learned self-connection weight β across all medium-scale datasets, sorted by edge homophily h. Positive β amplifies the node's own features; negative β suppresses them. Pearson correlation between β and h: $r = 0.006$, $p = 0.99$.